DATA ENGINEERING · ANALYTICS ENGINEERING

E-commerce dbt Project on Snowflake

A guided, screenshot-by-screenshot walkthrough: project configuration, data-quality tests, staging models, lineage, execution and test results.

dbt Core | Snowflake | SQL · Jinja | YAML | VS Code



OVERVIEW

Project at a glance

What was built, how it is organised and how it performed

ARCHITECTURE: RAW DATA TO BUSINESS TABLES



Data moves left to right. Raw tables are never modified; staging models clean them; marts combine the cleaned data into tables analysts can query directly.

PROJECT STRUCTURE (VS CODE EXPLORER)

```
ecommerce_dbt/
├─ models/
│  ├─ staging/   clean 1:1 views of raw data
│  │  ├─ src_raw.yml        declares raw sources
│  │  ├─ schema.yml         column tests
│  │  └─ stg_customers / stg_orders / stg_products .sql
│  └─ marts/     business-ready tables
│     ├─ fct_orders.sql
│     ├─ mart_category_sales.sql
│     └─ mart_customer_summary.sql
├─ tests/  assert_completed_order_revenue_positive.sql
├─ dbt_project.yml    project settings
├─ profiles.yml       Snowflake connection
└─ seeds · snapshots · macros · analyses
```

6 / 6
models built successfully

12.4 s
total dbt run time

9 / 10
data tests passing

1
test in ERROR (needs a fix)

WHAT EACH LAYER IS FOR

- **Sources** – the untouched raw tables loaded into Snowflake.
- **Staging** – rename, cast and trim. One model per raw table, kept as views so they are always fresh and cost no storage.
- **Marts** – joins and aggregations, stored as tables for fast queries (600, 5 and 120 rows).
- **Tests** – automatic quality checks that stop bad data from silently reaching reports.

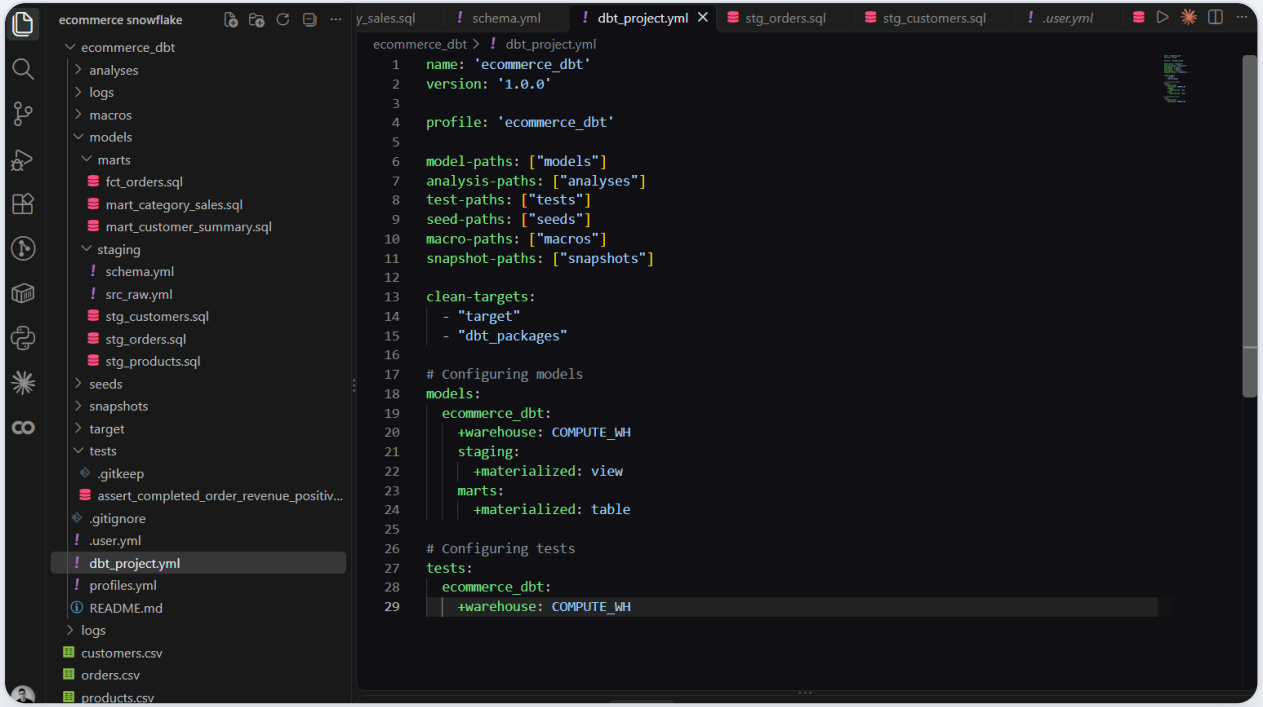
HOW TO READ THIS DOCUMENT

Follow the numbered pages in order. Each page shows the screenshot, a breakdown table, and why the step matters.

STEP 1

Project configuration

dbt_project.yml — the control centre of the project



SETTING BY SETTING

SETTING	VALUE	WHAT IT DOES
`name`	`ecommerce_dbt`	Identity of the project (used in selectors and docs).
`profile`	`ecommerce_dbt`	Entry in profiles.yml that holds the Snowflake credentials.
`model-paths`	`["models"]`	Folder dbt scans for models. Others point to analyses, tests, seeds, macros, snapshots.
`clean-targets`	`target, dbt_packages`	Folders removed by `dbt clean`.
`staging +materialized`	view	Staging models are stored as lightweight views.
`marts +materialized`	table	Marts are physically stored for fast reads.
`+warehouse`	`COMPUTE_WH`	Snowflake compute intended for models and tests.

WHAT IS HAPPENING

- dbt reads this file first on every command. It tells dbt **where files live** and **how to build them**.
- Configuration is applied by **folder**: everything in `models/staging` becomes a view, everything in `models/marts` becomes a table.
- This follows the standard layering pattern: cheap, always-fresh cleaning first; stored business tables last.

WHY IT MATTERS

- Changing one line here changes how a whole layer is built, so no per-file settings are needed.
- The `+` prefix means “apply to this folder and every sub-folder”.

PRO TIP

Keep **views for staging** and **tables for marts** until data volume or query cost says otherwise. Later you can move heavy marts to `incremental` to rebuild only new rows.

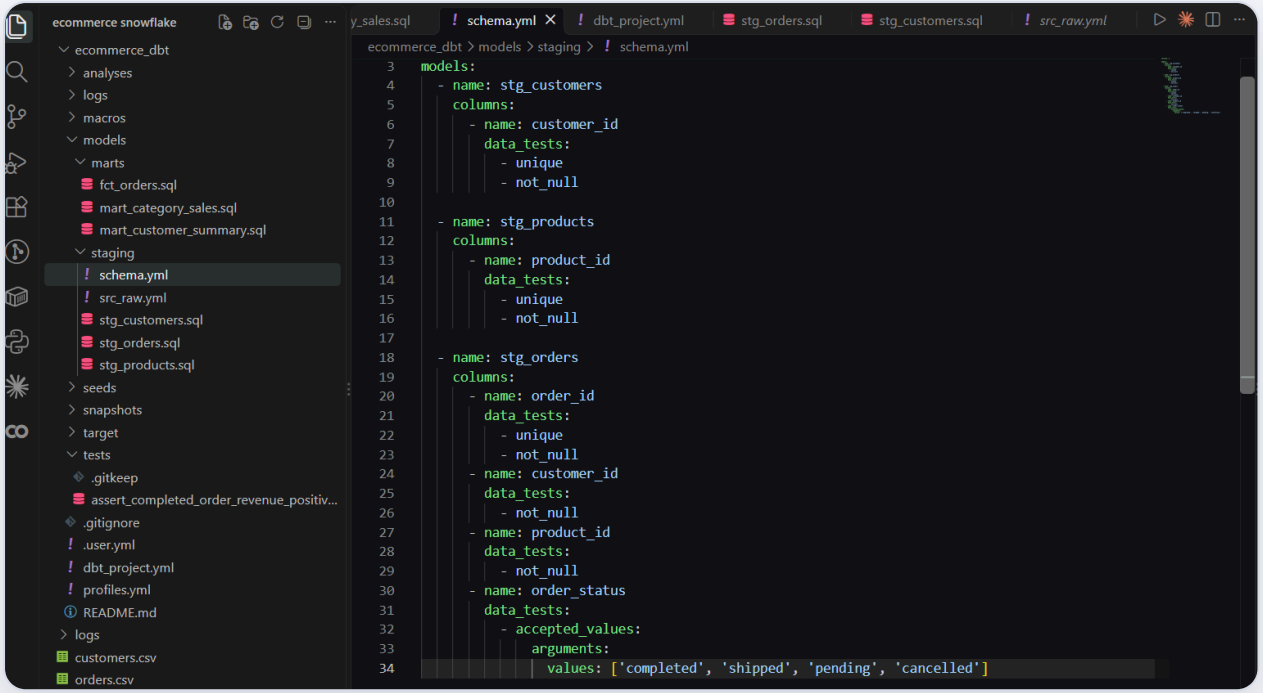
TRY IT IN YOUR TERMINAL

```
# check the Snowflake connection
dbt debug
# list models in a folder
dbt ls --select staging
```

STEP 2

Data-quality tests

schema.yml — rules every staging column must satisfy



TESTS DECLARED PER COLUMN

MODEL	COLUMN	TESTS
`stg_customers`	`customer_id`	unique, not_null
`stg_products`	`product_id`	unique, not_null
`stg_orders`	`order_id`	unique, not_null
`stg_orders`	`customer_id`	not_null
`stg_orders`	`product_id`	not_null
`stg_orders`	`order_status`	accepted_values — completed, shipped, pending, cancelled

WHAT IS HAPPENING

- Each `data_tests` entry is a **generic test**. dbt turns it into a SQL query that looks for violations.
- `unique` finds duplicated keys. `not_null` finds missing values. `accepted_values` finds anything outside the allowed list.
- A test **passes when its query returns zero rows**.
- These 9 column tests plus 1 custom SQL test give the total of **10 tests** you will see on the results page.

WHY IT MATTERS

- Primary keys must be unique, or later joins will silently multiply rows and inflate revenue.
- A restricted `order_status` list catches typos like “Complted” before they reach reports.

PRO TIP

The `arguments:` block under `accepted_values` is the newer syntax in recent dbt versions. It keeps test options clearly separated from test settings.

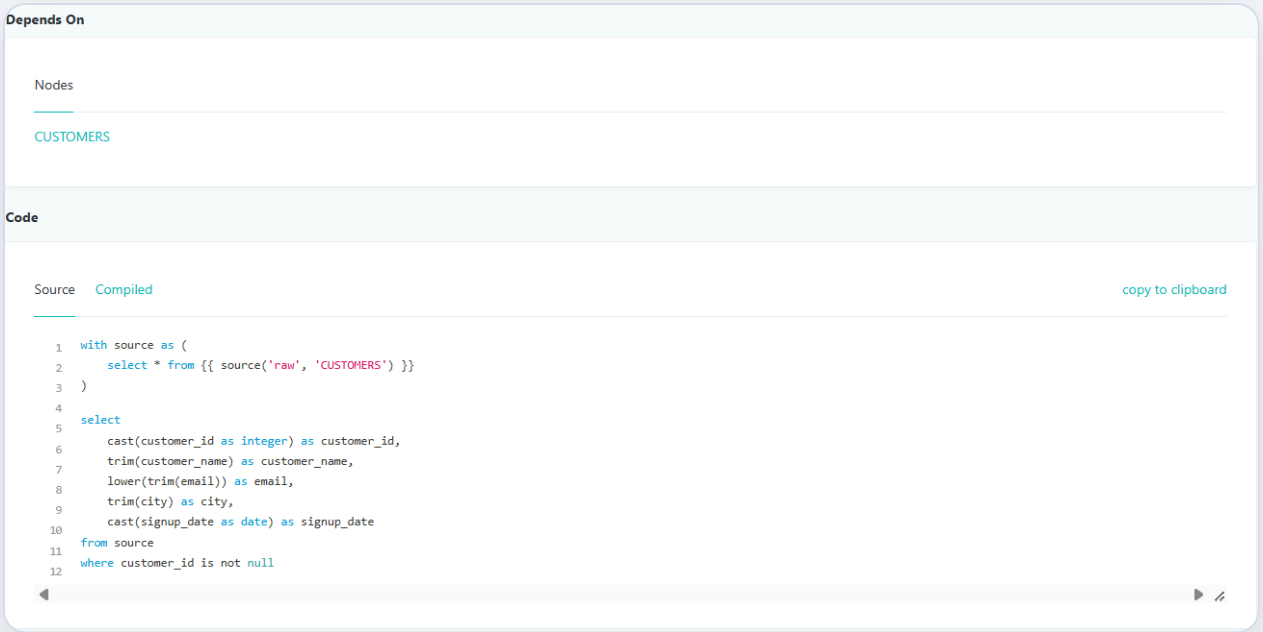
TRY IT IN YOUR TERMINAL

```
# run all tests on one model
dbt test --select stg_orders
# run only generic tests
dbt test --select test_type:generic
```

STEP 3

Staging model: stg_customers

Cleaning the raw CUSTOMERS table



COLUMN-BY-COLUMN TRANSFORMATIONS

COLUMN	SQL APPLIED	WHY
`customer_id`	`cast(... as integer)`	Numeric key that joins reliably.
`customer_name`	`trim()`	Removes stray leading/trailing spaces.
`email`	`lower(trim())`	Normalised for matching and de-duplication.
`city`	`trim()`	Consistent values for grouping.
`signup_date`	`cast(... as date)`	Real date type for time analysis.
filter	`where customer_id is not null`	Drops rows that have no key.

WHAT IS HAPPENING

- The docs page shows the **Depends On** graph: this view reads only the raw source `CUSTOMERS`.
- The query starts with a CTE named `source` that selects from `{{ source('raw','CUSTOMERS') }}`. Using `source()` instead of a hard-coded table name lets dbt draw lineage.
- The final `select` lists every column explicitly with its cleaned form.

WHY IT MATTERS

- Emails in mixed case would count the same customer twice; lower-casing prevents that.
- Explicit casts protect downstream models from raw text columns.

PRO TIP

Staging is the **only place that touches raw data**. Marts only read `ref('stg_...')`, so a raw-column rename is fixed in one file instead of everywhere.

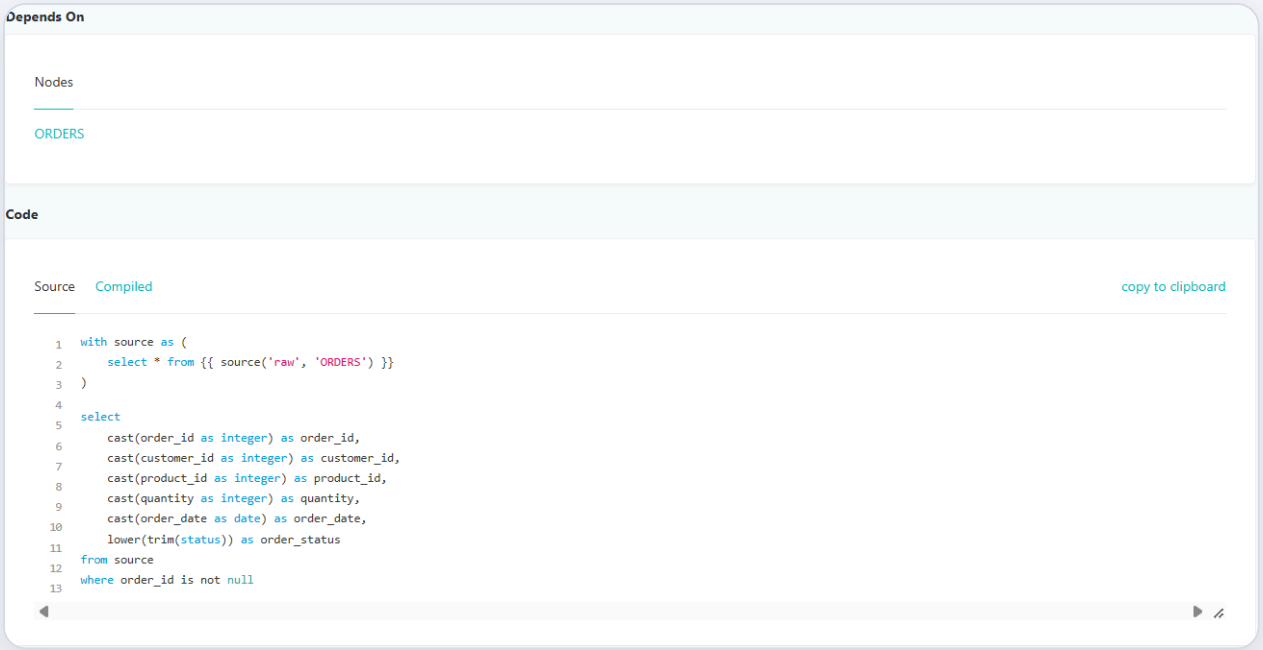
TRY IT IN YOUR TERMINAL

```
# build just this model
dbt run --select stg_customers
# test just this model
dbt test --select stg_customers
# preview its rows
dbt show --select stg_customers
```

STEP 4

Staging model: stg_orders

Cleaning the raw ORDERS table



COLUMN-BY-COLUMN TRANSFORMATIONS

COLUMN	SQL APPLIED	WHY
`order_id`	`cast(... as integer)`	Primary key of the order.
`customer_id`	`cast(... as integer)`	Foreign key to customers.
`product_id`	`cast(... as integer)`	Foreign key to products.
`quantity`	`cast(... as integer)`	Whole numbers for revenue maths.
`order_date`	`cast(... as date)`	Date type for trends and filters.
`status`	`lower(trim(status))`	Renamed to `order_status` and standardised.
filter	`where order_id is not null`	Drops rows that have no key.

WHAT IS HAPPENING

- Depends only on the raw source `ORDERS`.
- It carries both foreign keys, so later models can join orders to customers and to products.
- Status text is lower-cased and trimmed, so " Completed " becomes `completed` and matches the allowed list in schema.yml.

WHY IT MATTERS

- This is the model the `accepted_values` test protects.
- Revenue logic in marts can rely on clean `quantity` and status values.

PRO TIP

Staging is the **only place that touches raw data**. Marts only read `ref('stg_...')`, so a raw-column rename is fixed in one file instead of everywhere.

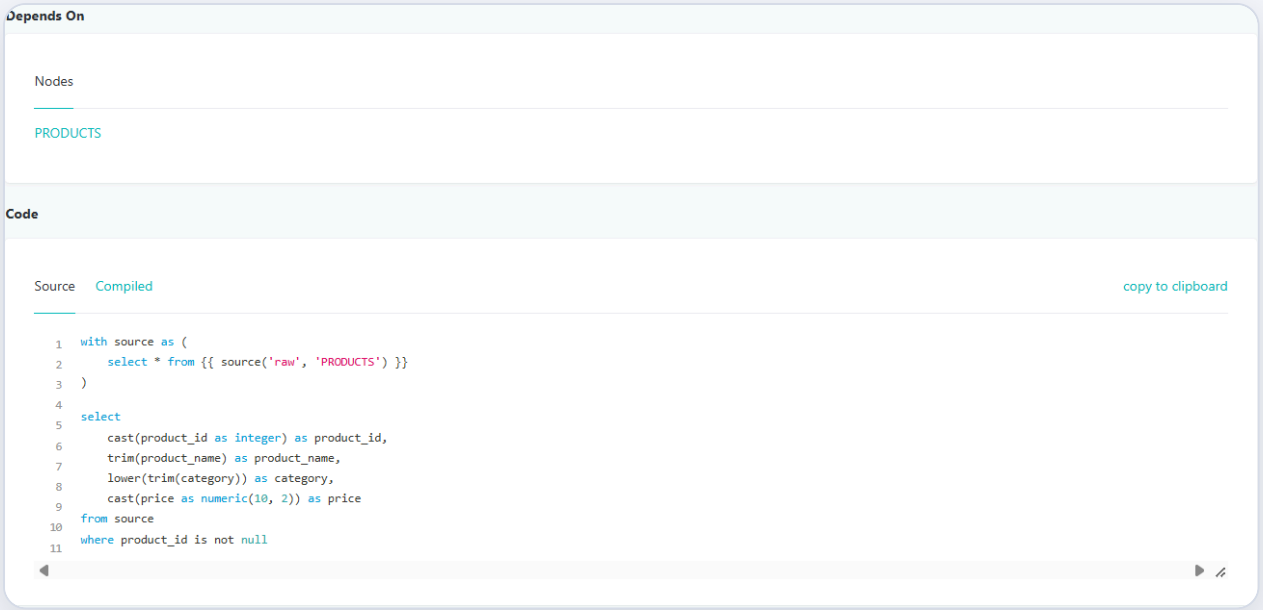
TRY IT IN YOUR TERMINAL

```
# build just this model
dbt run --select stg_orders
# test just this model
dbt test --select stg_orders
# preview its rows
dbt show --select stg_orders
```

STEP 5

Staging model: stg_products

Cleaning the raw PRODUCTS table



COLUMN-BY-COLUMN TRANSFORMATIONS

COLUMN	SQL APPLIED	WHY
`product_id`	`cast(... as integer)`	Primary key of the product.
`product_name`	`trim()`	Clean display name.
`category`	`lower(trim())`	Groups like “Books” and “books” together.
`price`	`cast(... as numeric(10,2))`	Up to 10 digits, 2 decimals: the right type for money.
filter	`where product_id is not null`	Drops rows that have no key.

WHAT IS HAPPENING

- Depends only on the raw source `PRODUCTS`.
- Category is normalised here because `mart_category_sales` groups by it later.
- The screenshot also shows the empty **Columns** table at the top: documenting columns would fill it in.

WHY IT MATTERS

- Using `numeric(10,2)` avoids floating-point rounding errors in money.
- Clean categories make the 5-row category sales mart accurate.

PRO TIP

Staging is the **only place that touches raw data**. Marts only read `ref('stg_...')`, so a raw-column rename is fixed in one file instead of everywhere.

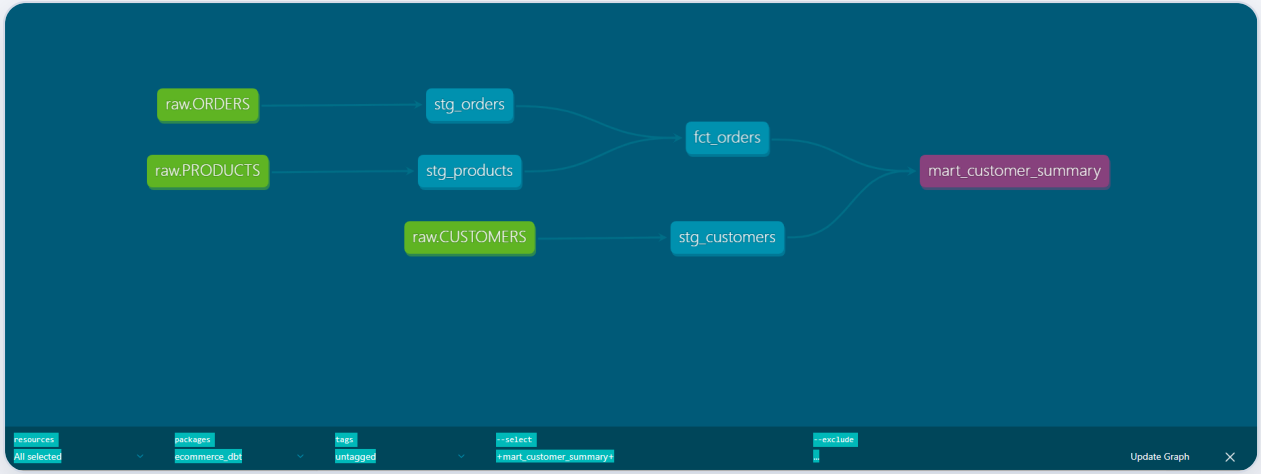
TRY IT IN YOUR TERMINAL

```
# build just this model
dbt run --select stg_products
# test just this model
dbt test --select stg_products
# preview its rows
dbt show --select stg_products
```

STEP 6

Lineage graph

How every model depends on the others



READING THE GRAPH

NODE	TYPE	READS FROM
`raw.ORDERS`	source	—
`raw.PRODUCTS`	source	—
`raw.CUSTOMERS`	source	—
`stg_orders`	view	`raw.ORDERS`
`stg_products`	view	`raw.PRODUCTS`
`stg_customers`	view	`raw.CUSTOMERS`
`fct_orders`	table	`stg_orders` + `stg_products`
`mart_customer_summary`	table	`fct_orders` + `stg_customers`

WHAT IS HAPPENING

- Green = sources, blue = models, purple = the final mart. Arrows show which model reads which.
- `fct_orders` is the fact table: orders enriched with product data.
- `mart_customer_summary` then adds customer information to produce one row per customer.
- The selector at the bottom, `+mart_customer_summary+`, filters the view to that model and everything around it. `mart_category_sales` is not shown because it sits on a different branch.

SELECTOR CHEAT SHEET

- `dbt run --select +mart_customer_summary` builds it with all its parents.
- `dbt run --select fct_orders+` builds it and everything that depends on it.
- `dbt run --select staging` builds only that folder.

PRO TIP

Lineage answers “what breaks if I change this?” before you change it. Check the downstream nodes first.

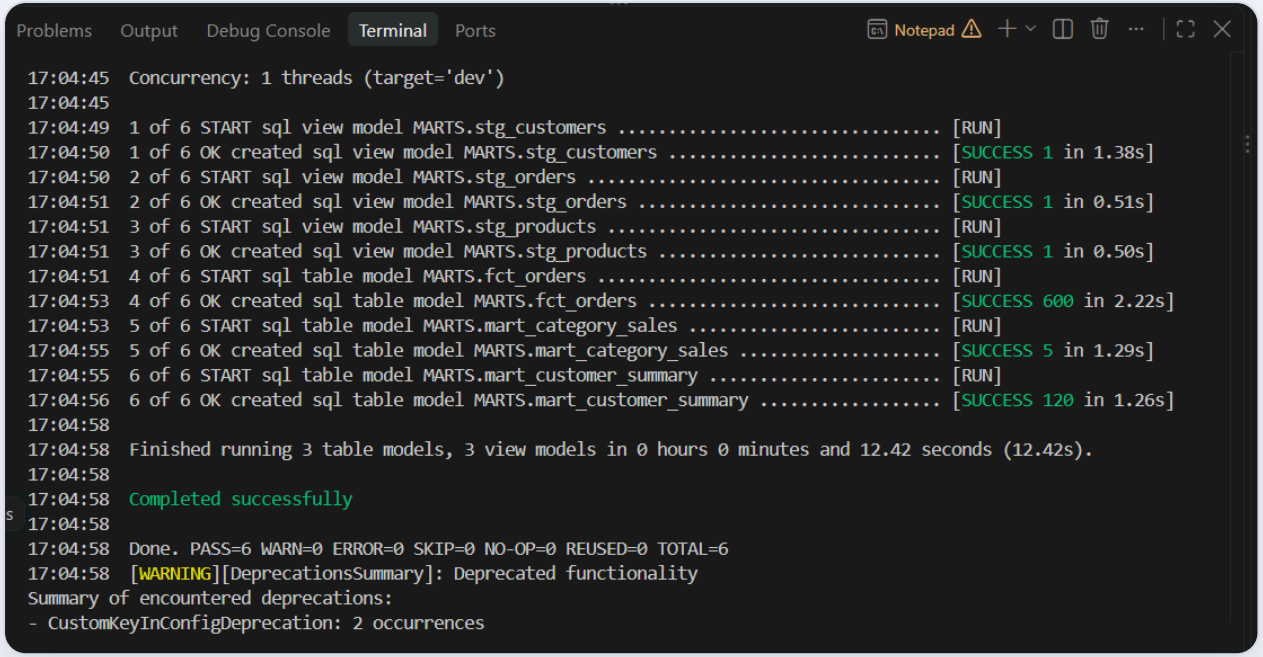
TRY IT IN YOUR TERMINAL

```
# generate the docs site
dbt docs generate
# open it and explore lineage
dbt docs serve
```

STEP 7

Building the models: dbt run

Six models created in Snowflake in dependency order



EXECUTION LOG

#	MODEL	TYPE	RESULT	TIME
1	`stg_customers`	view	created	1.38 s
2	`stg_orders`	view	created	0.51 s
3	`stg_products`	view	created	0.50 s
4	`fct_orders`	table	**600** rows	2.22 s
5	`mart_category_sales`	table	**5** rows	1.29 s
6	`mart_customer_summary`	table	**120** rows	1.26 s

WHAT IS HAPPENING

- dbt ran with **1 thread** on target `dev`, so models were built one after another in dependency order.
- Views are fast because only the query is stored. Tables take longer because the data is physically written.
- Final line: **PASS=6, WARN=0, ERROR=0**. Every model compiled and ran in Snowflake.

READING THE NUMBERS

- **600** orders in the fact table, **120** customers in the summary, **5** categories in the category mart.
- Quick sanity check: the customer summary should have the same number of rows as distinct customers.

WARNING TO FOLLOW UP

The yellow `CustomKeyInConfigDeprecation` notice appears **2 times**. It does not stop the run. The two `+warehouse` lines in dbt_project.yml are the likely cause. See the last page.

TRY IT IN YOUR TERMINAL

```
# build everything
dbt run
# build only staging
dbt run --select staging
# rebuild a table from scratch
dbt run --full-refresh
```

STEP 8

Testing the data: dbt test

10 tests executed: 9 passed, 1 error

Problems Output Debug Console **Terminal** Ports Notepad

```
17:05:48  1 of 10 PASS accepted_values_stg_orders_order_status__completed__shipped__pending__cancelled  [PASS in 1.28s
]
17:05:48  2 of 10 START test assert_completed_order_revenue_positive ..................... [RUN]
17:05:49  2 of 10 ERROR assert_completed_order_revenue_positive ..................... [ERROR in 0.31s]
17:05:49  3 of 10 START test not_null_stg_customers_customer_id ..................... [RUN]
17:05:49  3 of 10 PASS not_null_stg_customers_customer_id ..................... [PASS in 0.24s]
17:05:49  4 of 10 START test not_null_stg_orders_customer_id ..................... [RUN]
17:05:49  4 of 10 PASS not_null_stg_orders_customer_id ..................... [PASS in 0.31s]
17:05:49  5 of 10 START test not_null_stg_orders_order_id ..................... [RUN]
17:05:49  5 of 10 PASS not_null_stg_orders_order_id ..................... [PASS in 0.27s]
17:05:49  6 of 10 START test not_null_stg_orders_product_id ..................... [RUN]
17:05:50  6 of 10 PASS not_null_stg_orders_product_id ..................... [PASS in 0.44s]
17:05:50  7 of 10 START test not_null_stg_products_product_id ..................... [RUN]
17:05:50  7 of 10 PASS not_null_stg_products_product_id ..................... [PASS in 0.46s]
17:05:50  8 of 10 START test unique_stg_customers_customer_id ..................... [RUN]
17:05:51  8 of 10 PASS unique_stg_customers_customer_id ..................... [PASS in 0.32s]
17:05:51  9 of 10 START test unique_stg_orders_order_id ..................... [RUN]
17:05:51  9 of 10 PASS unique_stg_orders_order_id ..................... [PASS in 0.33s]
17:05:51  10 of 10 START test unique_stg_products_product_id ..................... [RUN]
17:05:51  10 of 10 PASS unique_stg_products_product_id ..................... [PASS in 0.30s]
17:05:52
17:05:52  Finished running 10 data tests in 0 hours 0 minutes and 7.91 seconds (7.91s).
17:05:52
```

RESULTS

TEST	STATUS	TIME
`accepted_values_stg_orders_order_status`	PASS	1.28 s
`assert_completed_order_revenue_positive`	ERROR	0.31 s
`not_null_stg_customers_customer_id`	PASS	0.24 s
`not_null_stg_orders_customer_id`	PASS	0.31 s
`not_null_stg_orders_order_id`	PASS	0.27 s
`not_null_stg_orders_product_id`	PASS	0.44 s
`not_null_stg_products_product_id`	PASS	0.46 s
`unique_stg_customers_customer_id`	PASS	0.32 s
`unique_stg_orders_order_id`	PASS	0.33 s
`unique_stg_products_product_id`	PASS	0.30 s

WHAT IS HAPPENING

- dbt ran **10 data tests** in 7.91 s. All 9 generic tests from schema.yml passed: keys are unique, required columns have no nulls, and statuses are valid.
- `assert_completed_order_revenue_positive` is a **singular test**: a custom SQL file in `tests/` that should return no rows when revenue is correct.

PASS VS FAIL VS ERROR

- **PASS** – the test query ran and returned zero bad rows.
- **FAIL** – the query ran and found bad rows (a data problem).
- **ERROR** – the query could not run at all (a code problem such as a wrong column name or object not found).

NEEDS ATTENTION

This test shows **ERROR, not FAIL**, so the data has not been judged yet. The SQL itself needs fixing first. Steps are on the next page.

TRY IT IN YOUR TERMINAL

```
# run just the failing test
dbt test --select assert_completed_order_revenue_positive
# run models and tests together
dbt build
```

NEXT STEPS

Findings & how to improve

Fix the one failing test, clean the warning, and harden the pipeline

1 · FIX THE ERROR

1. Run `dbt test --select assert_completed_order_revenue_positive` and read the red message.
2. Open the compiled SQL in `target/compiled/` and paste it into a Snowflake worksheet to see the exact error.
3. Check column names, `ref()` usage and that the model was built first. Common causes: a column that does not exist in `fct_orders`, or a hard-coded table name.
4. Re-run the test. It should now PASS or FAIL with real rows.

```sql
-- tests/assert_completed_order_revenue_positive.sql
-- example shape: adapt column names to fct_orders
select order_id, revenue
from {{ ref('fct_orders') }}
where order_status = 'completed'
  and revenue ⩽ 0
```

2 · CLEAN UP CONFIGURATION

- The run log shows models landing in the **MARTS** schema, including the staging views. A `+schema` setting per folder can separate layers.
- The Snowflake adapter documents the model-level warehouse option as `snowflake_warehouse`. The generic `+warehouse` key is the likely source of the 2 deprecation notices, and probably has no effect.
- Re-run `dbt run` afterwards to confirm the warning is gone.

```yaml
# dbt_project.yml
models:
  ecommerce_dbt:
    +snowflake_warehouse: COMPUTE_WH
    staging:
      +materialized: view
      +schema: staging   # optional
    marts:
      +materialized: table
```

3 · HARDEN THE PIPELINE

- **Test the raw sources** as well: staging filters out null keys, so staging tests can hide raw-data problems.
- Add `relationships` tests: `stg_orders.customer_id` must exist in `stg_customers`, same for products.
- Add `description:` to models and columns, then run `dbt docs generate` to fill the empty Columns tables.
- Use `dbt build` to run models and tests together in DAG order, and run it on every pull request in CI.

COMMAND CHEAT SHEET

`dbt debug`
Test the Snowflake connection and project setup.

`dbt run`
Build all models (views and tables).

`dbt test`
Run every data test.

`dbt build`
Run models and tests together in dependency order.

`dbt docs generate`
Create the documentation site and lineage graph.

`dbt clean`
Delete the target and dbt_packages folders.